详细设计说明书

```
RuankoDBMS/
├── CMakeLists.txt              # 🟢 [A/B级] 顶层构建脚本
├── README.md
│
├── client/                     # 🟡 [C级] 客户端模块 (对应 3.7 客户端管理)
│   ├── network/                # 负责与 Server 的 Socket 通信
│   └── ui/                     # Qt 图形化界面
│
├── server/                     # 🟢 [A/B级] 服务端 / 数据库核心引擎
│   ├── include/                # 【接口契约层】(只放 .h 文件)
│   │   ├── common/             # 🟢 [A/B级] ErrorCode, Constants 等全局定义。
│   │   ├── meta/               # 🟢 [A/B级] DatabaseMeta, TableMeta 元数据结构
体定义
│   │   ├── network/            # 🟡 [C级] 监听多客户端连接的 Socket 接口 (3.7)
│   │   ├── parser/             # 🟢 [A/B级] ISQLParser 接口与 AST 节点定义 (翻译
SQL)
│   │   ├── catalog/            # 🟢 [A/B级] ICatalogManager 接口 (管理系统元数
据) 数据字典管理器
│   │   ├── storage/            # 🟢 [A/B级] IStorageEngine 接口 (磁盘文件基础读
写) 存储引擎
│   │   ├── execution/          # 🟢 [A/B级] IExecutor 执行器核心接口 (调度中心)
│   │   │
│   │   │   # 以下均为 C 级选做项预留的独立接口目录
│   │   ├── index/              # 🟡 [C级] 索引管理接口 (3.6)
│   │   ├── transaction/        # 🟡 [C级] 事务管理接口 (3.8)
│   │   ├── integrity/          # 🟡 [C级] 完整性管理接口 (3.9)
│   │   ├── maintenance/        # 🟡 [C级] 数据库维护/备份还原接口 (3.10)
│   │   └── security/           # 🟡 [C级] 安全性与权限管理接口 (3.11)
│   │
│   ├── src/                    # 【业务实现层】(只放 .cpp 文件)
│   │   ├── parser/             # 🟢 实现 SQL 解析，考虑使用抽象语法树 (AST)
│   │   ├── catalog/            # 🟢 实现对 ruanko.db, .tb, .tdf 文件的解析与写入
│   │   ├── storage/            # 🟢 实现对 .trd 数据文件的 4 字节对齐读写
│   │   ├── execution/          # 🟢 实现各类执行器：执行调度器（核心发号施令者）
│   │   │   ├── CreateExecutor.cpp # 🟢 [A级] 创库/创表执行逻辑
│   │   │   ├── InsertExecutor.cpp # 🟢 [A级] 插入记录执行逻辑
│   │   │   ├── SelectExecutor.cpp # 🟢 [A级] 查询记录执行逻辑
│   │   │   ├── UpdateExecutor.cpp # 🟢 [B级] 更新记录执行逻辑
│   │   │   └── DeleteExecutor.cpp # 🟡 [C级] 删除相关(删库/删表/删记录)
│   │   │
│   │   ├── index/              # 🟡 [C级] 实现 B+Tree，操作 .tid 和 .ix 文件
│   │   ├── transaction/        # 🟡 [C级] 实现并发控制机制与 .log 日志回滚
```

```
│   │   ├── integrity/          # 🟡 [C级] 实现外键、唯一约束等校验，操作 .tic
│   │   ├── maintenance/        # 🟡 [C级] 实现文件打包备份与覆盖还原
│   │   └── security/           # 🟡 [C级] 实现登录鉴权拦截
│   │
│   └── main.cpp                # 🟢 [A/B级] 服务端统一启动入口（依赖注入的组装车间）
│
├── tests/                      # 🟢 [A/B级] 单元测试用例
└── data/                       # 🟢 [A/B级] 数据库物理文件存储根目录（程序运行时生成）
```

server/

服务端，数据库的核心部分

server/include/

该 **include** 模块下所有文件均为 **接口** ，这个模块下，只放 **虚函数**，作为接口被调用。这些类在其他对应的模块下被具体实现
命名规则： 驼峰命名，I开头，代表接口。

server/include/common/

common 模块下存储全局定义。

server/include/common/Constants.h

该文件依照 **3.12** 定义了 **全局变量**

说明	常量名	数据类型	值
目录常量	DBMS_ROOT	char*	"data/"
（数据库）名称最大长度	MAX_NAME_LEN	int	128
（数据库数据文件夹）路径最大长度	MAX_PATH_LEN	int	256
默认内存页大小(4KB)	PAGE_SIZE	int	4096

枚举

说明	枚举类名	常量量说明	常量名	常量值
数据库类型枚举	DatabaseType			
		系统	SYSTEM	0

说明	枚举类名	常量量说明	常量名	常量值
		用户	USER	1

说明	枚举类名	常量量说明	常量名	常量值
数据类型枚举 (对应 3.12.1 数据类型)	DatabaseType			
		系统	SYSTEM	0
		用户	USER	1

说明	枚举类名	常量量说明	常量名	常量值
数据库类型枚举	DataType			
			INTEGER	1
			BOOL	2
			DOUBLE	3
			VARCHAR	4
			DATETIME	5

说明	枚举类名	常量量说明	常量名	常量值
全局错误码 (团队统一抛出这些错误，方便前端判断)	ErrorCode			
			SUCCESS	0
		数据库不存在	ERR_DB_NOT_EXIST	1001
		表不存在	ERR_TABLE_NOT_EXIST	1002
		字段（列）不存在	ERR_FIELD_NOT_EXIST	1003
		主键冲突	ERR_DUPLICATE_PK	2001
		违反非空约束	ERR_NULL_CONSTRAINT_VIOLATION	2002
		违反 Check 约束	ERR_CHECK_CONSTRAINT_VIOLATION	2003

说明	枚举类名	常量量说明	常量名	常量值
		违反唯一约束	ERR_UNIQUE_CONSTRAINT_VIOLATION	2004
		语法错误	ERR_SYNTAX_ERROR	3001
		插入数据类型与定义不符	ERR_TYPE_MISMATCH	3002

server/include/meta/

meta 模块下存储元数据结构

server/include/meta/TableMeta.h

数据库基本信息块 (对应 ruanko.db 文件结构) 3.12.3
DatabaseBlock

结构体成员	数据类型	说明
name	CHAR[128]	数据库名称
type	BOOL	数据库类型：系统数据库，用户数据库
filename	CHAR[256]	数据库数据文件夹全路径，保存记录文件与日志文件。
crtime	DATETIME	创建时间

表格信息结构块 (对应 .tb 文件结构)3.12.5
TableBlock

结构体成员	数据类型	说明
name	CHAR[128]	表格名称
record_num	INTERGER	记录数
field_num	INTERGER	该表字段数
tdf	CHAR[256]	表格定义文件路径
tic	CHAR[256]	表格完整性文件路径
trd	CHAR[256]	表格记录文件路径
tid	CHAR[256]	表格索引文件路径
crtime	DATETIME	创建时间
mtime	INTERGER	最后修改时间

字段定义块 (对应 .tdf 文件结构)3.12.6
FieldBlock

结构体成员	数据类型	说明
order	INTERGER	字段顺序
name	CHAR[128]	字段名称
type	INTERGER	字段类型
param	INTERGER	字段类型参数（VARCHAR\CHAR）
mtime	DATETIME	最后修改时间
integrities	INTERGER	完整性约束信息

完整性约束块 (对应 .tic) 3.12.8
IntegrityBlock

结构体成员	数据类型	说明
name	CHAR[128]	约束名称
field	CHAR[128]	字段名称
type	INTERGER	约束类型
param	CHAR[256]	参数

索引块 (对应 .tid) 3.12.9
IndexBlock

结构体成员	数据类型	说明
name	CHAR[128]	索引名称
unique	BOOLE	是否唯一索引，true 为唯一索引，false 为非唯一索引
asc	BOOLE	排序方式：true 为升序 asc, false 为降序
field_num	INTEGER	字段数，最多可以保存 2 个
fields	CHAR[128][2]	字段值，最多可以保存 2 个
record_file	CHAR[256]	索引对应记录文件的路径
index_file	CHAR[256]	索引数据文件的路径

server/include/storage/

storage 模块封装所有文件读写操作

server/include/storage/IStorageEngine.h

- 物理文件周期管理
  - virtual bool createFile(const std::string& filePath) = 0;

        - 创建空的物理文件 (用于 CREATE TABLE 时生成 .tdf, .trd, .tic, .tid)
    - virtual bool deleteFile(const std::string& filePath) = 0;
        - 删除物理文件 (用于 DROP TABLE)
        -
    - virtual bool createDirectory(const std::string& dirPath) = 0;
        - 创建文件夹 (用于 CREATE DATABASE 时生成数据库专属目录)
        -
    - virtual bool deleteDirectory(const std::string& dirPath) = 0;
        - 删除文件夹 (用于 DROP DATABASE 时连同里面的文件一并清空)
        -
- 偏移量读写操作 (引入了偏移量,在数据查询中可以通过偏移量直接定位,降低了性能开销)
    - virtual long getFileSize(const std::string& filePath) = 0;
        - 获取当前文件的总字节数
        -
    - virtual bool readRaw(const std::string& filePath, long offset, int size, char* buffer) = 0;
        - 在指定文件的绝对偏移量 (offset) 处，读取 size 大小的二进制数据
        -
    - virtual bool writeRaw(const std::string& filePath, long offset, int size, const char* data) = 0;
        - 在指定文件的绝对偏移量 (offset) 处，覆盖写入 size 大小的二进制数据
        - 注意!是覆盖写入!
        -
    - virtual long appendRaw(const std::string& filePath, int size, const char* data) = 0;
        - 将数据直接追加到文件末尾，并返回写入成功后的【起始字节偏移量】
        - 返回的 offset 将直接交给 Index 模块去构建 B+ 树节点。

server/include/catalog/

server/include/catalog/ICatalogManager.h

- virtual bool hasTable(const std::string& tableName) = 0;
    - 验证表是否存在 (从 ruanko.db 或 .tb 中查询)
    -
- virtual Meta::TableBlock getTableMeta(const std::string& tableName) = 0;
    - 获取表的完整元数据 (包含记录数、各文件路径等)
    -
- virtual std::vector<Meta::FieldBlock> getFields(const std::string& tableName) = 0;
    - 获取某个表的所有字段定义 (从 .tdf 文件读取并按 order 排序)

-
- virtual bool createTable(const Meta::TableBlock& tableMeta, const std::vector<Meta::FieldBlock>& fields) = 0;
  - 执行 CREATE TABLE 时调用：
  - 由该模块负责在 ruanko.db 中注册，并生成对应的 .tb 和 .tdf 结构写入磁盘
  -
- virtual bool dropTable(const std::string& tableName) = 0;
  - 执行 DROP TABLE 时调用：注销表信息
  -
- virtual bool updateField(const std::string& tableName, const Meta::FieldBlock& field) = 0;
  - 动态修改字段时调用 (对应 ALTER TABLE)
  -
- virtual bool addField(const std::string& tableName, const Meta::FieldBlock& field) = 0;
  - 向 .tdf 文件追加新字段，并使 .tb 文件中的 field_num 加 1
  -
- virtual bool dropField(const std::string& tableName, const std::string& fieldName) = 0;
  - 从 .tdf 文件移除字段，并使 .tb 文件中的 field_num 减 1
  -
- virtual Meta::DatabaseBlock getDatabaseMeta(const std::string& dbName) = 0;
  - 获取单个数据库的元数据信息
  -
- virtual bool hasDatabase(const std::string& dbName) = 0;
  - 验证数据库是否存在 (查询 ruanko.db)
  -
- virtual std::vector<Meta::DatabaseBlock> getAllDatabases() = 0;
  - 获取系统中所有数据库的列表 (用于客户端展示)
  -
- virtual bool createDatabase(const Meta::DatabaseBlock& dbMeta) = 0;
  - 执行 CREATE DATABASE 时调用：在 ruanko.db 中追加记录，并由该函数内部调用 Storage 创建数据文件夹
  -
- virtual bool dropDatabase(const std::string& dbName) = 0;
  - 执行 DROP DATABASE 时调用：从 ruanko.db 中移除记录
  -
- virtual bool updateTableMeta(const std::string& tableName, const Meta::TableBlock& newTableMeta) = 0;

    - 更新表的宏观元数据 (用于 INSERT/DELETE 后更新记录数 record_num，或更新修改时间 mtime)
    -
- virtual std::vector<Meta::IntegrityBlock> getIntegrities(const std::string& tableName) = 0;
    - 获取某个表的所有完整性约束 (从 .tic 文件读取)
    -
- virtual std::vector<Meta::IndexBlock> getIndices(const std::string& tableName) = 0;
    - 获取某个表的所有索引定义 (从 .tid 文件读取)
    - q
- virtual bool addIndex(const std::string& tableName, const Meta::IndexBlock& indexMeta) = 0;
    - 添加一个新的索引 (对应 CREATE INDEX，负责写入 .tid 文件)
    -
- virtual bool dropIndex(const std::string& tableName, const std::string& indexName) = 0;
    - 删除一个索引 (对应 DROP INDEX)
    -
- virtual bool addIntegrity(const std::string& tableName, const Meta::IntegrityBlock& integrityMeta) = 0;
    - 添加一个新的完整性约束 (对应 ALTER TABLE ADD CONSTRAINT，负责写入 .tic 文件)
    -
- virtual bool dropIntegrity(const std::string& tableName, const std::string& integrityName) = 0;
    - 删除一个约束 (对应 ALTER TABLE DROP CONSTRAINT)
    -
- virtual bool useDatabase(const std::string& dbName) = 0;
    - 切换当前使用的数据库 (对应 USE database_name)
    -
- virtual std::string getCurrentDatabase() = 0;
    - 获取当前正在使用的数据库名称，供 Executor 拼接文件路径用
    -

server/include/executor/

server/include/executor/IExecutor.h

- virtual bool execute() = 0;
    - 统一的执行入口。

    - 返回 true 表示执行成功，返回 false 表示执行失败（如主键冲突、类型不匹配等）。
    - 具体的错误信息可以通过全局的 ErrorCode 机制向外抛出。

server/include/parser

语句解析

server/include/parser/ISQLParser.h

- virtual ASTNode* parse(const std::string& sql) = 0;
    - 核心接口：接收纯文本 SQL，返回结构化的 AST 节点指针。
    - 如果语法错误（比如少敲了括号），直接返回 nullptr。

server/include/parser/ASTNode.h

这个节点正在研究中,还没想清楚.....//////////

server/src

接口具体实现。实现类继承 include 模块下的类。具体实现类的头文件和源文件放一起

server/src/storage/

server/src/storage/StorageEngineImpl

public:

- 物理文件生命周期管理
- bool createFile(const std::string& filePath) override;
- bool deleteFile(const std::string& filePath) override;
- bool clearFile(const std::string& filePath) override;
- 目录管理
- bool createDirectory(const std::string& dirPath) override;
- bool deleteDirectory(const std::string& dirPath) override;
- 绝对偏移量读写
- long getFileSize(const std::string& filePath) override;
- bool readRaw(const std::string& filePath, long offset, int size, char* buffer) override;
- bool writeRaw(const std::string& filePath, long offset, int size, const char* data) override;
- long appendRaw(const std::string& filePath, int size, const char* data) override;

server/src/parser/

server/src/parser/SQLParserImpl

public:

- 核心解析接口的实现
- ASTNode* parse(const std::string& sql) override;
- std::string trim(const std::string& str);
- std::string toUpperCase(const std::string& str);